SIVF: GPU-Resident IVF Index for Streaming Vector Analytics

Dongfang Zhao (dzhao@uw.edu)

Abstract

GPU-accelerated Inverted File (IVF) index is one of the industry standards for large-scale vector analytics but relies on static VRAM layouts that hinder real-time mutability. Our benchmark and analysis reveal that existing designs of GPU IVF necessitate expensive CPU-GPU data transfers for index updates, causing system latency to spike from milliseconds to seconds in streaming scenarios. We present SIVF, a GPU-native index that enables high-velocity, in-place mutation via a series of new data structures and algorithms, such as conflict-free slab allocation and coalesced search on non-contiguous memory. SIVF has been implemented and integrated into the open-source vector search library, Faiss. Evaluation against baselines with diverse vector datasets demonstrates that SIVF reduces deletion latency by orders of magnitude compared to the baseline. Furthermore, distributed experiments on a 12-GPU cluster reveal that SIVF exhibits near perfect linear scalability, achieving an aggregate ingestion throughput of 4.07 million vectors/s and a deletion throughput of 108.5 million vectors/s.

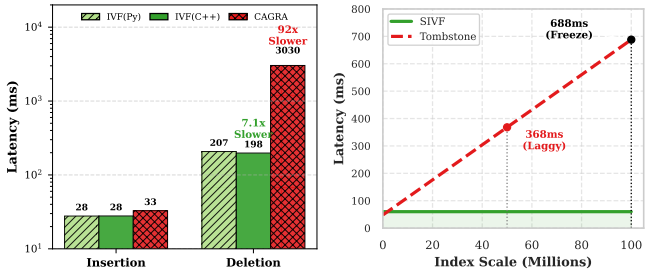
1 Introduction

Real-world vector analytics applications, ranging from search engines to dynamic retrieval-augmented generation (RAG) of large language models (LLMs), increasingly operate on streaming data where timeliness is critical [30, 38]. These systems necessitate a sliding-window model where expired vectors must be invalidated promptly as new vectors arrive to maintain bounded memory footprint. However, existing GPU-accelerated approximate nearest neighbor (ANN) indices are predominantly optimized for static, write-once-read-many workloads [33, 55].

To quantify the gap between insertion and eviction performance, we conducted benchmarks with the industry-standard Faiss library [21] and the state-of-the-art graph index CAGRA (cuVS) [39] on an NVIDIA RTX 6000 GPU hosted at the Chameleon Cloud [24]. We utilized the SIFT1M dataset [23] to simulate a realistic sliding-window scenario, measuring the wall-clock latency of ingesting 10,000 new vectors and evicting the oldest 10,000.

As illustrated in Figure 1(a), we observe a drastic asymmetry between insertion and physical deletion. While GPU parallelism accelerates insertion to ~28 ms for IVF, the eviction process is significantly slower. For Faiss IVF, physical deletion spikes the latency to ~200 ms (~7.1× slower) due to the contiguous memory layout necessitating expensive data shifting. The situation is catastrophic for graph-based indices: CAGRA incurs a deletion latency of over 3,000 ms (~92× slower) as the topology requires reconstruction to maintain connectivity.

A common alternative is *lazy deletion* (aka tombstone), which marks vectors as invalid to avoid immediate physical movement. However, this strategy merely defers the cost. As shown in Figure 1(b), tombstone mechanisms suffer from poor scalability due to the $O(N)$ complexity of garbage collection (compaction). Based on our microbenchmarks, the compaction pause scales linearly with index size. While negligible at small scales, the latency is projected



(a) Physical deletion overhead. **(b) Tombstone scalability trap.**

Figure 1: The dilemma of mutability on GPUs.

to exceed 360 ms at 50 million vectors and reach nearly 700 ms at 100 million vectors. These results confirm a fundamental limitation: current GPU indices lack an efficient mutability mechanism that is both low-latency (unlike physical deletion) and scalable (unlike tombstones), necessitating a new indexing method ideally with a constant cost, as represented by SIVF that we will present.

Our analysis of the Faiss source code [7] further reveals that this performance asymmetry stems from a rigid class hierarchy that enforces a fallback to host-side processing on CPUs. In the library's architecture, the data eviction interface is defined in the abstract base class `faiss::Index` via the `remove_ids` virtual method. While CPU-based implementations override this method to provide efficient in-memory deletion logic (e.g., `memmove`), the GPU counterparts inherit the base implementation which lacks native support. Consequently, current GPU indices rely on a *CPU-GPU Roundtrip* pattern for eviction. That is, to delete even a single vector, the system must transfer the entire index state from VRAM to host memory, perform the compaction on the CPU, and re-upload the modified index to the device. This heavy I/O burden saturates the PCIe bus and prevents the system from utilizing the GPU's compute throughput, capping the maximum sustainable ingestion rate in streaming scenarios.

The lack of a GPU-resident eviction operator is not an oversight, but a consequence of how GPU IVF indices are engineered for throughput. To maximize memory coalescing and bandwidth utilization, inverted lists are typically stored in pre-allocated contiguous buffers. In this setting, naive tombstone deletion is hard to make practical. First, tombstones provide only logical deletion and do not reclaim space, so under a sliding-window workload dead entries accumulate and lists grow, forcing queries to scan and filter more invalid slots unless compaction is performed. Second, compaction and slot reuse in VRAM require bulk data movement and careful synchronization with concurrent queries and updates under the GPU memory model. In addition, standard inverted file (IVF) indices lack an ID-to-Location mapping. Without a reverse index, locating a specific ID for deletion requires scanning all inverted lists, an operation with $O(N)$ complexity that negates the benefits of GPU acceleration. These constraints make naive GPU eviction prohibitively inefficient.

To resolve these limitations, we propose SIVF, a GPU-resident indexing method that supports high-throughput mutability through four specialized device-side algorithms. First, to enable nonblocking writes, we design a *Lock-Free Ingestion* protocol (Algorithms 1 & 2) that manages a pool of fixed-size slabs. This protocol utilizes `atomic Compare-And-Swap` for slot reservation and speculative head updates, allowing concurrent threads to append to linked lists without global locks. Second, to guarantee data consistency under CUDA's relaxed memory model, we apply a strict publication ordering; the kernel issues device-wide memory fences (`__threadfence()`) before committing validity bits, ensuring readers never observe partially initialized payloads. Third, to mitigate the latency of pointer chasing, we propose *Warp-Cooperative Search* (Algorithm 3). By aligning the slab capacity ($C = 32$) with the hardware warp width, this algorithm enables threads to cooperatively load and evaluate non-contiguous memory blocks while preserving coalesced access patterns. Finally, we address deletion by maintaining a GPU-resident Address Translation Table along with lazy eviction (Algorithm 4), allowing SIVF to decouple logical removal from physical data movement, which makes eviction to a constant-time operation.

SIVF has been implemented and integrated into Faiss [7] as a new index through the `GpuIndexSIVF` class. Our implementation leverages CUDA for device-side kernels and employs a dual-view memory management strategy to maintain the consistency between host and device states. All new algorithms as well as key data structures are encapsulated within a `SlabManager` class that abstracts the complexity of dynamic memory operations on the GPU.

We have evaluated SIVF against a comprehensive set of baselines within Faiss, including CAGRA (cuVS) [39], IVF [21], Flat [7], HNSW [35], NSG [9], and LSH [5]. Our evaluation was carried out on six datasets representing a variety of modalities, dimensions, and scalability: synthetic [7], Deep1B [51], SIFT1M [23], T2I-1B [43], GIST1M [23], and DINO10B [1]. Experimental results show that SIVF achieves up to 1765x reduction in deletion latency compared to CPU fallback baselines and improves ingestion throughput by 36x to 120x compared to existing GPU indices. Furthermore, distributed evaluations on a 12-GPU cluster demonstrate that SIVF exhibits near-perfect linear scalability, reaching an aggregate deletion throughput of 108.5 million vectors/s and 159.3K vectors/s for high-fidelity search. In end-to-end sliding window scenarios, SIVF delivers a 163x to 262x speedup with a negligible memory overhead less than 0.8 percent, effectively closing the gap between static and streaming vector analytics performance.

In summary, this paper makes the following contributions:

- We identify the CPU-GPU bottleneck in existing GPU vector indices when being used for sliding-window scenarios, demonstrating that the lack of native deletion support renders them unsuitable for streaming vector analytics.
- We propose a new indexing method, namely SIVF, that synthesizes a series of GPU-optimized data structures and algorithms for streaming vector analytics. This design enables $O(1)$ time complexity for deletion and constant-time insertion overhead with negligible memory overhead.
- We implement SIVF in the state-of-the-art vector search library Faiss and evaluate SIVF against a broad spectrum of baselines including GPU CAGRA, GPU Flat, GPU IVF, HNSW, NSG, and LSH on 6 representative datasets and a 12-GPU cluster. The results demonstrate that SIVF achieves up to 1765x reduction in deletion latency, up to 120x faster ingestion compared to existing GPU indices, and exhibits near perfect linear scalability in distributed environments reaching 108.5 million vectors/s for deletion and 4.07 million vectors/s for ingestion on 12 GPUs. These advancements transform vector search from a static retrieval task into an integral component of real-time vector data analytics.

2 Related Work

Optimization of Graph-Based Indices. Graph-based indices remain the state-of-the-art for high-recall retrieval. To address performance bottlenecks in production, frameworks like VSAG [56] optimize memory access patterns and parameter tuning, while SOAR [44] improves indexing structures. Of note, CAGRA [39] establishes a new state-of-the-art for static graph construction and search throughput on GPUs; however, it is fundamentally designed for write-once-read-many workloads and lacks native support for the efficient, fine-grained in-place deletions required by streaming applications. Navigational efficiency is a key optimization target; SHG [12] introduces a hierarchical graph with shortcuts to bypass redundant levels, and probabilistic routing methods [34] have been proposed to enhance traversal. Several works focus on distance metric optimizations: ADA-NNS [22] utilizes angular distance guidance to filter irrelevant neighbors, DADE [6] accelerates comparisons via data-aware distance estimation in lower dimensions, and HSCG [42] adapts the Monotonic Relative Neighbor Graph for cosine similarity using hemi-sphere centroids. Efforts to improve graph construction and maintenance include LIGS [4], which employs locality-sensitive hashing (LSH) to simulate proximity graphs for faster updates, and CSPG [52], which crosses sparse proximity graphs. Addressing robustness, Hua et al. [14] propose dynamically detecting and fixing graph hardness for out-of-distribution queries. MIRAGE-ANNS [46] attempts to bridge the gap between incremental and refinement-based construction. Furthermore, theoretical frameworks like Subspace Collision [49] provide guarantees on result quality via clustering-based indexing. Collectively, while these approaches maximize navigational efficiency for static data, they fundamentally rely on complex edge dependencies that are prohibitively expensive to maintain on GPUs. Unlike SIVF, they lack the architectural support for $O(1)$ lock-free mutation, rendering them unsuitable for high-churn streaming workloads where sub-millisecond update latency is critical.

Attribute-Filtered and Hybrid Search. The integration of structured constraints with vector analytics has led to a surge in Filtered ANN research. Li et al. [27] provide a comprehensive taxonomy and benchmark of these methods. For range filtering, UNIFY [31] introduces a segmented inclusive graph to support pre-, post-, and hybrid filtering strategies. Dynamic environments are addressed by DIGRA [20], which uses a multi-way tree structure for dynamic graph indexing, and RangePQ [54], which offers a linear-space indexing scheme for range-filtered updates. Peng et al. [40] propose dynamic segment graphs to handle mixed streams of data and queries. Regarding specific constraints, Wang et al. [47] present a robust framework for general attribute constraints. Engels et al. [8]

focus on window filters, while WoW [48] develops a window-graph based index to handle window-to-window incremental construction. However, these contributions primarily optimize algorithmic filtering logic rather than the underlying storage architecture. They remain constrained by the static memory layouts of standard GPU indices, whereas SIVF fundamentally redesigns the VRAM management to enable the high-throughput, in-place mutability that is a prerequisite for such dynamic operations.

Quantization and Scalability. To manage high-dimensional data at scale, quantization and system-level optimizations are critical. RaBitQ [11] and its extended version [10] provide theoretical error bounds for bitwise quantization with flexible compression rates. SymphonyQG [13] integrates quantization codes directly with graph structures to reduce memory access overhead, while LoRANN [19] utilizes low-rank matrix factorization for compression. In terms of system architecture, Tagore [29] leverages GPUs for accelerating graph index construction. For distributed environments, HARMONY [50] employs a multi-granularity partition strategy to balance load. WebANNS [32] enables efficient search within web browsers via WebAssembly. From a theoretical perspective, Indyk and Xu [18] analyze the worst-case performance limits of popular implementations. Finally, DARTH [2] introduces declarative recall targets through adaptive early termination to balance performance and result quality. While these techniques reduce storage footprints or distribute load, they predominantly operate on static memory layouts that are brittle under frequent updates. SIVF addresses a complementary challenge: it provides the dynamic memory substrate that allows these scalable architectures to support real-time streams without succumbing to VRAM fragmentation or locking overheads.

GPU Acceleration. Recent HPC work highlights how careful kernel and data-layout design can unlock GPU performance across data-intensive workloads. FZ-GPU demonstrates a fully parallel compression pipeline with both warp-aware bitwise operations and shared-memory efficiency [53]. For sparse linear algebra, SpMV designs reduce preprocessing overhead while improving load balance and memory access locality [3], and GPU-aware preconditioning further emphasizes locality and coalescence as first-order concerns on modern GPU architectures [25]. Beyond compute kernels, GPU-enabled asynchronous checkpoint caching and prefetching treat GPU memory as a first-class tier to improve end-to-end throughput for I/O-heavy workflows [36]. Format choices and composition are also critical for irregular workloads, as shown by automatic sparse format composition for SpMM that avoids expensive autotuning while delivering strong performance [41]. Another line of work focuses on reliability, debugging, and cluster-scale GPU management, which together shape how GPU-centric systems are built and operated. GPU-FPX provides low-overhead floating-point exception tracking for NVIDIA GPUs [28], FPBOXer improves scalable input generation for triggering floating-point exceptions in GPU programs [45], and FloatGuard extends whole-program exception detection to AMD GPUs and highlights cross-vendor portability concerns [37]. At the application and cluster level, SIMCoV-GPU studies multi-node, multi-GPU acceleration for irregular agent-based simulations [26], FASOP automates fast search for near-optimal transformer parallelization on heterogeneous GPU clusters [17], and serverless platforms motivate GPU sharing and scheduling mechanisms such as ESG and FluidFaaS [15, 16]. However, applying these general-purpose HPC optimizations to vector search remains nontrivial due to the irregular memory access patterns of IVF traversal. SIVF bridges this gap by tailoring these micro-architectural principles, specifically warp-aligned slabs and lock-free synchronization, to construct the first GPU-resident index capable of sustaining high-velocity updates without compromising retrieval integrity.

3 Design and Implementation of SIVF

This section presents the design and implementation of SIVF for high-concurrency updates on GPUs. We first introduce the Slab-based Dynamic Memory Allocator (SDMA) in Section 3.1, which represents each IVF list as a linked chain of fixed-capacity slabs with per-list head pointers $H[\cdot]$ and per-slab metadata

$$(\texttt{next}, \texttt{valid\_count}, \texttt{validity\_bitmap}),$$

summarized in Algorithm 1. Building on SDMA, we describe the parallel ingestion (Section 3.2) in which each thread inserts one vector by reserving a slot via `atomicCAS` on `valid_count` and publishing visibility by setting a validity bit after `__threadfence()` (Algorithm 2). We then present a warp-cooperative search (Section 3.3) that assigns one warp to one query and evaluates one slab per traversal step by consulting the validity bitmap before reading payloads (Algorithm 3). We detail lazy eviction (Section 3.4), which performs constant-time deletion by looking up coordinates in the address table $\mathcal{T}$ and atomically clearing the corresponding bitmap bit (Algorithm 4). We demonstrate the theoretical properties of SIVF in Section 3.5 and discuss implementation details in Section 3.6.

Data Model and Assumptions. To ensure $O(1)$ address resolution without hashing collisions, SIVF operates on a dense identifier space $v_{id} \in [0, N_{max})$, where N_{max} is the pre-configured capacity. In production environments, sparse or non-integer external IDs (e.g., 64-bit UUIDs) are mapped to this internal dense range via a lightweight host-side registry. Furthermore, SIVF enforces strong consistency for data updates via a "delete-then-insert" semantic: overwriting an existing v_{id} involves atomically clearing its existing validity bit (logical deletion) before the new payload becomes visible. This design ensures that the validity bitmap serves as the single source of truth for searchers.

3.1 Slab-based Memory Management

We propose a Slab-based Dynamic Memory Allocator (SDMA) that replaces monolithic, variable-length inverted lists with linked chains of fixed-size blocks on GPU. SDMA pre-allocates a contiguous slab pool; each slab stores up to C vectors and a lightweight metadata header $M = \langle n_{next}, b_{valid}, \text{cnt} \rangle$, where n_{next} is the next-slab pointer, b_{valid} is a C-bit validity bitmap, and cnt tracks the number of active vectors. We set $C = 32$ to match the GPU warp size. Each IVF list ℓ is represented by a head pointer $H[\ell]$ to the first slab in its chain. Algorithm 1 summarizes these core primitives.

Allocation draws a fresh slab id from a global stack pointer P_{top} via $s = \texttt{atomicSub}(P_{top}, 1) - 1$, initializes its metadata, and links it to the list head $H[\ell]$ using an atomic compare-and-swap (CAS) operation. Insertion for a vector id v_{id} first resolves the target slab

Algorithm 1 Core operations of SDMA

Global State: slab pool metadata and data, per-list head pointers $H[\cdot]$, stack pointer P_{top}, address table $\mathcal{T}$
Input: vector identifier v_{id}, vector x, list assignment ℓ
procedure INSERT(v_{id}, x, ℓ)
    $s \leftarrow H[\ell]$
    while s is invalid **or** no free slot exists in slab s **do**
        $s_{\text{new}} \leftarrow$ `atomicSub`$(P_{\text{top}}, 1) - 1$
        Initialize metadata s_{new} with $b_{\text{valid}} = 0$, $n_{\text{next}} = s$
        Update $H[\ell]$ to s_{new} via `atomicCAS`
        $s \leftarrow H[\ell]$
    end while
    Reserve a free slot o in slab s via atomic update of c_{valid}
    Write x to slab data$[s][o]$
    Atomically set bit o in b_{valid}
    $\mathcal{T}(v_{\text{id}}) \leftarrow \langle s, o \rangle$
end procedure
procedure DELETE(v_{id})
    $\langle s, o \rangle \leftarrow \mathcal{T}(v_{\text{id}})$
    if $\langle s, o \rangle \neq$ `INVALID` **then**
        old_map $\leftarrow$ Atomically clear bit o in b_{valid} of slab s
        if bit o was set in *old_map* **then**
            $\mathcal{T}(v_{\text{id}}) \leftarrow$ `INVALID`
            old_cnt $\leftarrow$ `atomicSub`(&slab_meta$[s]$.cnt, 1)
            if *old_cnt* == 1 **then**
                Atomically push s back to P_{top} stack
                slab_meta$[s]$.$b_{\text{valid}} \leftarrow 0$
            end if
        end if
    end if
end procedure

s, reserves a free slot o, writes the payload, and finally commits visibility by atomically setting bit o in b_{valid}.

To support $O(1)$ *physical* deletion without expensive garbage collection, SDMA maintains an address table $\mathcal{T}$ mapping each id to its physical coordinate $\mathcal{T}(v_{id}) = \langle s, o \rangle$. The deletion operation performs a direct lookup and atomically clears bit o in b_{valid}. Unlike tombstone-based designs, SDMA performs immediate reclamation: it atomically decrements the slab's occupancy counter cnt. If cnt drops to zero, the slab is instantly recycled by pushing it back to the global free stack P_{top}, making it available for future insertions.

3.2 Lock-free Parallel Ingestion

SIVF uses a fully parallel ingestion kernel in which each CUDA thread inserts one vector into its assigned IVF list ℓ. Each list is a singly linked chain of fixed-capacity slabs, with head pointer $H[\ell]$ and per-slab metadata `valid_count`, `validity_bitmap`, and `next`. Insertion follows a reserve-write-publish protocol. A thread first reads the current head slab $h = H[\ell]$ and attempts to reserve a slot by incrementing `slab_meta[`h`].valid_count` using `atomicCAS`; the reserved slot index is the old counter value c. On success, the thread writes the payload to `slab data[`h`][`c`]` and the id to `slab_ids[`h`][`c`]`, updates the address table $\mathcal{T}(v_{id}) \leftarrow \langle h, c \rangle$, then executes `__threadfence()` and sets the corresponding validity bit using `atomicOr`. The validity bit is the only publication signal read by search, so this ordering ensures that a reader observing the bit set never sees partially initialized payload or a missing table entry. If the head slab is absent or full, the thread allocates a new slab id from the global pool via `atomicSub` on P_{top}, initializes its metadata (`valid_count`, `validity_bitmap`, `next`), executes `__threadfence()`, and attempts to publish it as the new head with `atomicCAS(&H[`ℓ`], h, s_new)`. If head publication fails under contention (i.e., another thread updated the head first), the thread immediately reclaims the allocated slab to the global free list to prevent memory leaks and retries the operation. If the slab pool is exhausted, the insertion fails fast for that element and returns an error to the caller, which can throttle the update stream or retry later, rather than silently dropping updates. The full per-thread protocol is described in Algorithm 2.

3.3 Coalesced Search on Non-Contiguous Slabs

SIVF stores each inverted list as a linked chain of fixed-capacity slabs, which replaces contiguous scans with traversal via `next`. To retain high GPU throughput, we use a warp-cooperative search kernel that matches slab capacity to the warp width ($C = 32$). The kernel launches one warp per query. The warp first stages the query vector into shared memory, then probes *nprobe* coarse lists returned by the quantizer. For each probed list ℓ, the warp traverses the slab chain starting from $s = H[\ell]$. At each slab s, lane j consults `slab_meta[`s`].validity_bitmap` and evaluates slot j only if the corresponding bit is set. If set, the lane loads the payload from `slab data[`s`][`j`]`, retrieves the label from `slab_ids[`s`][`j`]`, computes the distance to the shared query, and updates a small per-lane top-k structure kept in registers. After traversal, lanes write their local top-k candidates to shared memory and one lane merges them into the final top-k result for the query. To ensure termination under unexpected corruption, traversal is bounded and the kernel breaks on self-loops. Algorithm 3 summarizes the procedure.

3.4 Lazy Eviction via Address Translation Table

SIVF supports constant-time deletion by combining an Address Translation Table (ATT) with bitmap-based lazy eviction. The ATT $\mathcal{T}$ maps each identifier u to its physical coordinate in the slab pool as a 64-bit value $\mathcal{T}[u] = (\text{idx}_{slab} \ll 32) \mid \text{idx}_{slot}$, or `INVALID` if absent. Given $\mathcal{T}[u]$, a deletion thread decodes $(\text{idx}_{slab}, \text{idx}_{slot})$ and clears the corresponding bit in the slab validity bitmap using an atomic read-modify-write (`atomicAnd` with $mask = \sim (1 \ll \text{idx}_{slot})$). To handle duplicates and races, the kernel uses the pre-update bitmap value to detect a $1 \rightarrow 0$ transition. Bookkeeping and reclamation occur strictly on this transition: the thread decrements the slab's `valid_count` and marks $\mathcal{T}[u]$ as `INVALID`. If the decremented `valid_count` drops to zero, implying the slab has become fully empty, the thread atomically pushes the slab index back to the global free stack P_{top}. This immediate reclamation strategy allows SIVF to reuse memory for new insertions without requiring heavy background compaction. Algorithm 4 summarizes the procedure.

Algorithm 2 Lock-free ingestion protocol in SIVF

Input: vector x, identifier v_{id}, list assignment ℓ
Global: head array $H[\cdot]$, slab_meta, slab_data, free list, stack pointer P_{top}, address table $\mathcal{T}$
$attempts \leftarrow 0$
while $attempts <$ MAX_INSERT_ATTEMPTS **do**
  $attempts \leftarrow attempts + 1$
  $h \leftarrow H[\ell]$
  if $h \neq -1$ **then**
    $c \leftarrow$ slab_meta$[h]$.valid_count
    if $c < C$ **then**
      if `atomicCAS`(&slab_meta$[h]$.cnt, $c, c+1$) **then**
        Write payload x to slab_data$[h][c]$
        Write identifier to slab_id_buffer$[h][c]$
        $\mathcal{T}(v_{id}) \leftarrow \langle h, c \rangle$
        `__threadfence()`
        `atomicOr`(&slab_meta[h].valid_bitmap, mask)
        return
      end if
    end if
  end if
  $t \leftarrow$ `atomicSub`$(P_{top}, 1)$
  if $t \leq 0$ **then**
    `atomicAdd`$(P_{top}, 1)$
    return
  end if
  $s_{new} \leftarrow$ free_list$[t-1]$
  Set slab_meta$[s_{new}]$.valid_count $\leftarrow 1$
  Set slab_meta$[s_{new}]$.validity_bitmap $\leftarrow 0$
  Set slab_meta$[s_{new}]$.next $\leftarrow h$
  `__threadfence()`
  if `atomicCAS`$(\&H[\ell], h, s_{new}) == h$ **then**
    Write payload x to slab_data$[s_{new}][0]$
    Write identifier to slab_id_buffer$[s_{new}][0]$
    $\mathcal{T}(v_{id}) \leftarrow \langle s_{new}, 0 \rangle$
    `__threadfence()`
    `atomicOr(&slab_meta[s_new].validity_bitmap)`
    return
  else
    $t_{ret} \leftarrow$ `atomicAdd`$(P_{top}, 1)$
    free_list$[t_{ret}] \leftarrow s_{new}$
  end if
end while

Algorithm 3 Warp-cooperative search in SIVF

Input: queries Q, probed list ids $coarse\_ids$, head array $H[\cdot]$
Input: slab metadata and data, buffer $slab\_ids$, k, $nprobe$
Output: top-k distances and labels for each query
for each query index q in parallel **do**
  Launch one warp for query q
  Copy $Q[q]$ into shared memory
  Initialize per-thread local top-k lists in registers
  for $p = 0$ to $nprobe - 1$ **do**
    $\ell \leftarrow coarse\_ids[q, p]$
    if ℓ is invalid **then**
      continue
    end if
    $s \leftarrow H[\ell]$
    while s is valid and traversal bound not exceeded **do**
      $md \leftarrow slab\_meta[s]$
      if $md.next = s$ **then**
        break
      end if
      if $j < 32$ and $md.validity\_bitmap[j]$ is set **then**
        Load vector $\mathbf{x}_{s,j}$ from slab data
        Compute $d(\mathbf{q}, \mathbf{x}_{s,j})$
        $id \leftarrow slab\_ids[s, j]$
        Insert (d, id) into local top-k
      end if
      $s \leftarrow md.next$
    end while
  end for
  Write local top-k lists to shared memory
  One thread merges 32 lists and writes final top-k outputs
end for

3.5 Theoretical Analysis

3.5.1 Correctness. We prove three properties. The first establishes the correctness of parallel ingestion in Algorithm 2. The second shows that search in Algorithm 3 is safe under concurrent ingestion and deletion. The third proves that lazy eviction in Algorithm 4 is linearizable with a clear linearization point.

We use the same state as the pseudocode. For each list id ℓ, $H[\ell]$ stores the head slab id. Each slab s has metadata `slab_meta[`s`]` including `next`, `valid_count`, and a 32-bit `validity_bitmap`. A slot (s, o) is logically present if and only if bit o in the bitmap is 1. Payloads and ids are stored in `slab_data[`s`][`o`]` and `slab_ids[`s`][`o`]`. The address table $\mathcal{T}$ maps a vector id u to a coordinate (s, o) encoded in `coord`, or `INVALID`.

Theorem 3.1 (Parallel ingestion is safe and linearizable)**.** *Consider any concurrent execution of Algorithm 2. For every insertion that returns successfully, there exists a unique slot* (s, o) *such that: (i) the insertion operation writes the payload and id into that slot, then sets bit* o *in* `validity_bitmap` *exactly once, (ii) once the bit is set, the slot contains a fully initialized payload and id, and (iii) the insertion can be linearized at the atomic bit set operation* `atomicOr` *that makes the slot visible.*

Proof. We first argue the uniqueness of the chosen slot for a successful insertion.

In the existing head case, the code reads

$$c = \texttt{slab\_meta}[h].\texttt{valid\_count},$$

and attempts to reserve index c using an atomic compare-and-swap (CAS). Since the CAS operation atomically verifies that the count is still c before incrementing it to $c+1$, only one thread can succeed for any specific snapshot of the counter. Therefore, among concurrent contenders for the same slab h, at most one insertion obtains a given index c. In the new slab case, the thread publishes its new slab as the head using `atomicCAS(&H[ℓ], h, s_new)`; only one thread can win this publication for a fixed old head value h, and

Algorithm 4 Lazy eviction with slab-wise reclamation in SIVF

```
Input: deletion identifiers U = {u_1, ..., u_m}
Global: Address Table T, slab metadata, free list, stack pointer P_top
for each u in U in parallel do
    coord ← T[u]
    if coord == INVALID then
        continue
    end if
    idx_slab ← coord ≫ 32
    idx_slot ← coord & (2^32 − 1)
    mask ←  ~ (1 ≪ idx_slot)
    old_map ← atomicAnd(slab[idx_slab].bitmap, mask)
    if ((old_map ≫ idx_slot) & 1) == 1 then
        old_cnt = atomicSub(&slab_meta[idx_slab].cnt, 1)
        T[u] ← INVALID
        if old_cnt == 1 then
            t ← atomicAdd(P_top, 1)
            free_list[t] ← idx_slab
            slab_meta[idx_slab].bitmap ← 0
        end if
    end if
end for
```

the winner uses slot $o = 0$ in its newly allocated slab. Thus, every successful insertion selects exactly one slot (s, o).

We next argue that the chosen slot is initialized before it becomes visible. In both code paths, after reserving the slot, the writer performs the payload write to `slab_data[s][o]` and the id write to `slab_ids[s][o]`, then writes $\mathcal{T}(v_{id}) \leftarrow \langle s, o \rangle$, executes `__threadfence()`, and only after the fence performs `atomicOr` to set bit o in the bitmap. The memory consistency semantics of `__threadfence()` ensure that all prior global writes are committed to memory before any subsequent atomic operation by the same thread becomes visible to other threads. Therefore, any observer that sees bit o as set is guaranteed to see the fully initialized payload and id.

Finally, we justify linearizability of a successful insertion with the atomic bit set as the linearization point. The membership predicate used throughout the design is exactly the bitmap bit. Before the `atomicOr`, the bit is 0, so the slot is logically absent to any search warp. After the `atomicOr` takes effect, the bit is 1, so the slot is logically present. Since `atomicOr` executes atomically on the bitmap word, the state transition $0 \rightarrow 1$ occurs at a single instant. At that instant, the slot becomes visible and, by the fence ordering established above, it is valid. Thus, the insertion is linearizable. □

Theorem 3.2 (Search safety under concurrent ingestion and deletion)**.** *In any concurrent execution of Algorithms 2, 3, and 4, every payload and id read performed by Algorithm 3 is fully initialized.*

Proof. Algorithm 3 reads a slot (s, j) only if it first observes that bit j in `slab_meta[s].validity_bitmap` is set. When the search observes the bit set, the slot must have been published by a successful insertion. By Theorem 3.1, the publication point is the atomic bit set in Algorithm 2, and the payload and id writes occur before the fence, which ensures they complete before the bit set. The same memory consistency guarantee implies that once the bit set is visible to a search warp, the earlier payload and id writes are also visible. Therefore, the search cannot observe a partially initialized payload or id.

Deletion does not write payloads or ids. Algorithm 4 only clears the validity bit and, upon slab reclamation, resets the bitmap to 0 before returning the slab to the free pool. Hence, even if a search races with a delete or a subsequent slab reuse, the primary guard is the bitmap state. If the search sees the bit as 0, it skips the slot. If it sees the bit as 1, it reads a payload and id that were fully initialized before the corresponding bitmap set operation (whether from the original insertion or a reuse). In neither case does the search read corrupted or partially written data.

Algorithm 3 also terminates because its traversal is bounded, and it includes an explicit self-loop guard (`md.next == s`) before following `next`. Concurrent insertions can add a new head slab, but they do not require the search to revisit already visited slabs within the traversal bound. □

Theorem 3.3 (Lazy eviction is linearizable and makes deleted vectors invisible)**.** *Consider any concurrent execution of Algorithms 3 and 4. For any delete request on u such that $\mathcal{T}[u] \neq$ `INVALID` and decodes to (idx_{slab}, idx_{slot}), define*

$$mask = \sim(1 \ll idx_{slot}).$$

Then the atomic operation old_map ← atomicAnd() in Algorithm 4 constitutes the linearization point for logical deletion. After this atomic operation takes effect, the slot becomes logically absent from the search space. Repeated deletes of the same id are idempotent and safe.

Proof. If $\mathcal{T}[u] =$ `INVALID`, Algorithm 4 performs no state change and the delete is linearized at the `INVALID` check.

Otherwise, the delete decodes `coord` into $(\text{idx}_{slab}, \text{idx}_{slot})$, computes *mask*, and executes

$$old\_map \leftarrow \texttt{atomicAnd}(\text{slab\_meta}[\text{idx}_{slab}].\text{bitmap}, mask)$$

to clear the corresponding bit. Since `atomicAnd` is an atomic read-modify-write instruction on the GPU, the transition of the validity bit from 1 to 0 (or 0 to 0) occurs at a single, indivisible instant. We designate this instant as the linearization point.

Algorithm 3 uses the validity bit as the sole membership predicate. Because the search warp reads the bitmap atomically before accessing any payload, any search that observes the bitmap after the linearization point will see the bit as 0, skip the slot, and exclude the vector from the results. Physical payload reclamation is not required for correctness because logical membership is fully determined by the validity bit.

To handle duplicates, Algorithm 4 uses the return value `old_cnt` to detect if it was the specific thread that caused the $1 \rightarrow 0$ transition. If so, it performs bookkeeping (e.g., updating counters). If the bit was already 0, the operation is effectively a no-op on the logical state. Thus, the deletion mechanism is idempotent. □

3.5.2 Time Complexity. We analyze the time complexity relative to the total number of vectors N, the number of lists n_{list}, and dimensionality D. For ingestion, SIVF guarantees strict $O(1)$ time complexity as it requires only atomic counter updates and pointer

manipulation to append data, independent of the current list size. Similarly, deletion achieves $O(1)$ complexity by utilizing the Address Translation Table for constant-time lookup followed by a single atomic bitmap invalidation, completely eliminating the need for memory compaction or data movement. For search, the complexity remains $O(nprobe \cdot (N/n_{list}) \cdot D)$, where $nprobe$ is the user-configured number of lists to scan; while the linked-slab layout introduces minor pointer-chasing overhead, it preserves the asymptotic complexity class of inverted file search.

3.5.3 Space Complexity. The total space complexity is dominated by the vector payload, scaling linearly as $O(N \cdot D)$. The structural overhead consists of the Address Translation Table ($O(N)$, 8 bytes per entry) and slab metadata. Since metadata is amortized over the fixed slab capacity ($C = 32$), its footprint is negligible compared to high-dimensional payloads. Regarding memory efficiency, unlike dynamic arrays that typically reserve up to 2× capacity to amortize resizing costs, SIVF grows in fine-grained increments of single slabs. Although lazy deletion introduces sparse internal fragmentation (unused slots within active slabs), the immediate reclamation of fully empty slabs bounds this waste, ensuring the total memory footprint remains proportional to the active dataset size.

3.6 System Implementation

We implement SIVF as a fully integrated extension to the GPU components of the widely-adopted Faiss library [7]. Our implementation represents a significant systems engineering effort, involving approximately 13,000 lines of code as of January 2026. The majority of this codebase consists of optimized C++ and CUDA system implementation (spanning the core Faiss integration, slab memory management, and distributed MPI runtime), while the remaining lines comprise Python and Bash utility scripts for experimental orchestration. We have open-sourced the complete implementation and evaluation scripts on GitHub.

The core architecture centers on the `GpuIndexSIVF` class, a direct subclass of Faiss's `GpuIndex`. This class integrates our custom `SlabManager` to orchestrate dynamic memory on the GPU. Unlike standard static models, `SlabManager` maintains a monolithic memory pool and a device-side free list stack, enabling both warp-aligned allocation and $O(1)$ slab recycling without host intervention. To support high-concurrency operations, we implemented specialized GPU-native kernels for insertion, deletion, and search (encapsulated in `SIVFAppend`, `SIVFDelete`, and `SIVFSearch`). These operators interact with lock-free metadata structures via atomic state transitions to ensure data consistency during simultaneous read and write access.

Integrating these dynamic components into the static resource management of Faiss presented multiple challenges. Unlike standard Faiss indices which rely on flat arrays managed through the `DeviceTensor` class, SIVF implements its own allocator to minimize fragmentation overhead. We reconcile this by overriding the virtual `addImpl` and `searchImpl` methods, allowing SIVF to serve as a drop-in replacement in existing pipelines. Furthermore, our design adheres to the `GpuResources` context provided by Faiss.

3.7 Distributed Scale-out Architecture

To support large-scale datasets exceeding single-device memory capacity, we extended SIVF with a shared-nothing, data-parallel distributed architecture.

The global dataset is partitioned into disjoint shards, distributed across P GPU workers. Unlike model-parallel approaches that split inverted lists across devices (which incurs heavy synchronization overheads during ingestion), SIVF employs *Data Sharding*. Incoming vectors are routed to workers via a deterministic partitioning capabilities (e.g., round-robin or hash-based routing). This allows each GPU to ingest data into its local SIVF index independently, enabling the aggregate ingestion throughput to scale linearly with the cluster size, as evidenced in our evaluation (Section 4.7).

Query processing follows a *Scatter-Gather* pattern. The client broadcasts the query vector to all workers via MPI. Each worker performs a warp-cooperative search on its local shard to retrieve the top-k candidates. A global reduction step (implemented via `MPI_Gather` or tree-based reduction) merges the partial results to produce the final global top-k. For deletion, since SIVF maintains a dense ID space, deletion requests are broadcast to all workers. Each worker checks its local Address Translation Table (ATT). Due to the disjoint partitioning, the target ID exists on at most one worker, which then executes the lock-free lazy eviction locally.

4 Evaluation

4.1 Experimental Setup

To ensure a comprehensive evaluation, we adopt a two-tiered baseline strategy. First, our primary baseline is the standard Faiss GPU IVF, which serves as a direct architectural counterpart; this allows us to isolate the performance gains specifically attributable to our proposed memory management innovations, excluding confounding factors from differing algorithmic paradigms. Second, to position SIVF within the broader indexing landscape, we in Section 4.6 compare against representative non-IVF indices, including GPU CAGRA [39] (dynamic graph), GPU Flat (brute-force), HNSW [35] (dynamic graph), NSG [9] (static graph), and LSH [5] (hashing).

Most experiments were conducted on a bare-metal node from the Chameleon Cloud testbed [24] (CHI@UC site). The platform is equipped with dual-socket Intel Xeon Gold 6126 CPUs (Skylake microarchitecture), providing a total of 24 physical cores (48 threads with Hyper-Threading) clocked at 2.60 GHz. The host memory consists of 192 GiB of RAM. For acceleration, the node features an NVIDIA Quadro RTX 6000 GPU with 24 GB of GDDR6 VRAM. The system runs on a Ubuntu 24.04 environment with the CUDA toolkit installed: Driver ver. 560.35.05 and CUDA ver. 12.6. Each reported data point represents the average of at least three independent executions. Error bars are omitted in figures where the standard deviation is negligible to maintain visual clarity.

4.2 Microbenchmarks

4.2.1 Ingestion. We evaluate ingestion throughput against the standard Faiss GPU IVFFlat baseline, varying database size (N_B: 1M–4M) and cluster count (n_{list}: 1024–16384). As shown in Figure 2, SIVF achieves consistent performance advantages with up to 2.65× speedup.

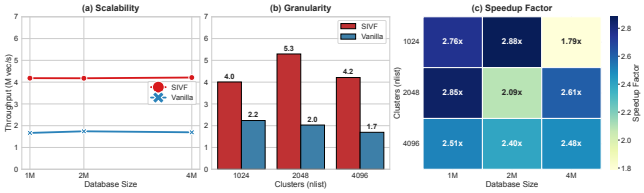


Figure 2: Microbenchmark performance of vector ingestion.

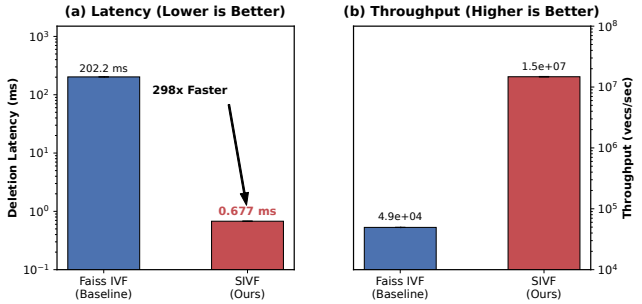


Figure 3: Microbenchmark performance of vector deletion.

As shown in Fig. 2a, SIVF maintains a stable throughput of ≈4.2M vec/s as N_B increases, significantly outperforming the baseline (≈1.7M vec/s). This stability validates our lock-free SlabManager, which ensures $O(1)$ insertion cost independent of index occupancy, thereby avoiding the contiguous memory resizing overheads inherent to the baseline. While the baseline throughput degrades monotonically as n_{list} increases (dropping from 2.2M to 1.7M vec/s), Fig. 2b shows that SIVF exhibits a performance sweet spot at $n_{list} = 2048$, peaking at 5.3M vec/s. At this optimal granularity, SIVF achieves a 2.65× speedup over the baseline. Even at high $n_{list} = 4096$, where scattered memory writes typically hinder performance, SIVF maintains robust throughput at 4.2M vec/s, sustaining a 2.47× advantage over the baseline's 1.7M vec/s. As shown in Fig. 2c, SIVF outperforms the baseline across all configurations, with speedups typically ranging from 2.4× to 2.9×. Even under maximum contention (4M vectors, $n_{list} = 1024$), SIVF retains a 1.79× advantage, confirming the efficiency of the non-blocking, slab-based pipeline for streaming scenarios.

4.2.2 Deletion. We evaluate the efficiency of the vector deletion mechanism in SIVF by comparing it against the standard Faiss GPU IVF baseline. The experiment measures latency and throughput when removing a batch of 10,000 vectors from a populated index of one million 128-dimensional vectors.

Figure 3 illustrates the performance comparison between the two methods. The results indicate that the baseline Faiss implementation incurs a substantial deletion latency of approximately 202.2 ms. In stark contrast, SIVF completes the same batch deletion operation in an average of 0.68 ms. This represents a massive speedup of 298.5×. Correspondingly, deletion throughput surges from 4.9×10^4 vec/s in the baseline to nearly 1.5×10^7 vec/s in SIVF.

4.2.3 Parameter Sensitivity and Ablation Study. We evaluate system robustness by sweeping three parameters: (i) vector capacity factor (*maxvec_factor*, or *mv*), (ii) slab pool redundancy (*slab_factor*, or *sl*), and (iii) deletion batch size (b). These control logical headroom, pre-allocated memory for bursts, and the trade-off between amortization and freshness. Figures 4 and 5 summarize the results.

Figures 4 and 5 illustrate the impact of pre-allocation strategies and batch sizes on system performance. Insertion throughput correlates positively with memory provisioning, peaking at 3.23M vec/s with a *maxvec_factor* of 1.5 and larger batches, as generous pre-allocation minimizes dynamic resizing and contention. While deletion throughput is sensitive to batching under tight constraints (dropping to 1.08M vec/s), increasing the *slab_factor* or *maxvec_factor* effectively decouples performance from resource limitations, stabilizing throughput at ≈1.7M vec/s. End-to-end latency remains robust across all settings, with insertions ranging from 3.09–3.69 ms and deletions achieving sub-millisecond performance (0.58–0.93 ms). This 3×–5× speed advantage for deletions stems from SIVF's lock-free bitmap mechanism, while the observed latency reduction with larger batches confirms that amortizing kernel launch overheads is a key driver for high-velocity streaming mutations.

4.3 Real-world Datasets

We benchmark SIVF using four popular real-world datasets representing diverse modalities and varying degrees of cluster skewness: Deep1B [51] (96 dimensions, deep features, imbalance factor $\mathcal{I} = 1.23$), SIFT1M [23] (128 dimensions, vision, $\mathcal{I} = 1.24$), T2I-1B [43] (200 dimensions, multimodal/RAG, $\mathcal{I} = 1.21$), and GIST1M [23] (960 dimensions, high-dim features, $\mathcal{I} = 1.76$). The high imbalance factor of GIST1M, in particular, stresses the allocator's ability to handle non-uniform distributions. While the subset size (1M vectors) fits within GPU memory, these experiments validate performance within the *active window* of streaming applications. Section 4.4 will report their stability under high-churn workloads.

As shown in Figure 6, SIVF dominates ingestion scalability across all dimensions. On Deep1B, SIVF achieves a peak throughput of 4.38M vec/s (120× speedup over the baseline's 36.4K vec/s). This advantage persists on SIFT1M (3.78M vec/s, 105×) and T2I-1B (2.91M vec/s, 84×). Even on high-dimensional GIST1M, SIVF maintains 852.7K vec/s (36× speedup), confirming that our pre-allocated slab architecture effectively saturates GPU bandwidth by eliminating dynamic resizing overhead.

Figure 7 highlights the critical performance divergence. Baseline latency scales poorly due to CPU-GPU roundtrips, varying from 1.2s (Deep1B) to 11.8s (GIST1M). In contrast, SIVF's in-place bitmap updates decouple latency from dimension, maintaining sub-millisecond performance (< 0.9 ms) across all datasets. Notably on GIST1M, SIVF reduces latency from 11,843 ms to 0.89 ms, a 13,307× improvement, validating its feasibility for real-time streams.

Figure 8 compares query throughput across diverse datasets. The results demonstrate that SIVF fundamentally overcomes the performance bottleneck typical of dynamic indexing without compromising retrieval accuracy. On Deep1B, SIVF outperforms the baseline by 2.07x (59.8K vs 28.9K QPS). On SIFT1M, it retains a 1.5x lead (40.9K vs 26.7K QPS), while on T2I-1B, it provides competitive performance reaching 10.8K QPS. While microbenchmarks in sparse scenarios (e.g., $n_{list} = 16384$) may show efficiency ratios

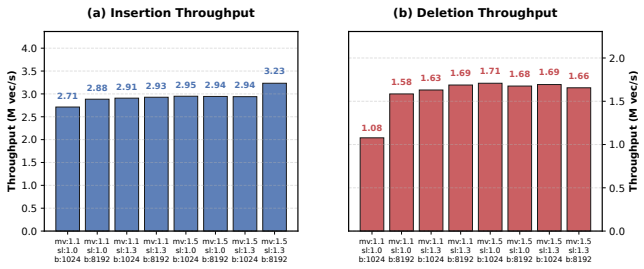


Figure 4: Throughput sensitivity analysis.

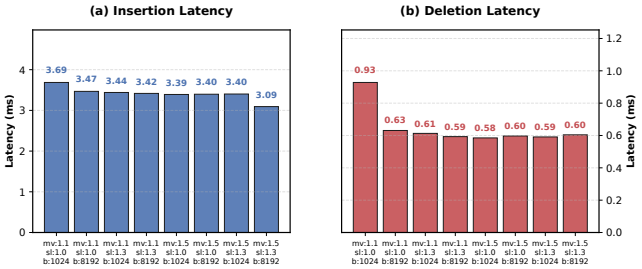


Figure 5: Latency sensitivity analysis.

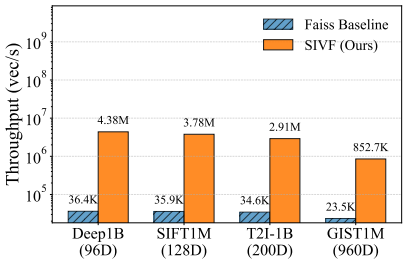


Figure 6: Ingestion throughput.

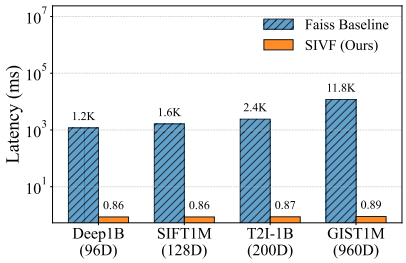


Figure 7: Deletion latency.

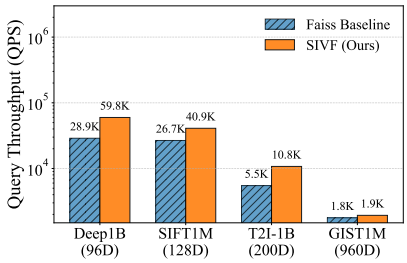


Figure 8: Search performance (QPS).

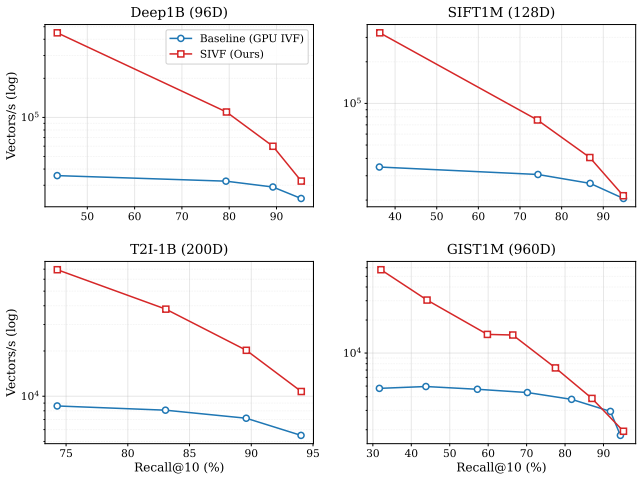


Figure 9: Throughput-Recall Pareto frontier analysis across four datasets (Deep1B, SIFT1M, T2I-1B, and GIST1M).

between 0.40x and 0.52x due to the physical trade-off of pointer chasing, SIVF demonstrates that query performance on real world datasets is highly adjustable through the n_{list} and n_{probe} parameters. By optimizing the coarse quantizer granularity, such as setting $n_{list} = 8192$ for GIST1M and $n_{list} = 4096$ for T2I-1B, SIVF maintains parity or superior query throughput even as search reaches high recall regimes.

Figure 9 illustrates the trade-off between query throughput and retrieval quality measured by Recall@10. The results confirm that SIVF provides a strictly superior operational envelope compared to the contiguous baseline across all tested dimensions. First, SIVF achieves strict recall parity, reaching identical maximum recall targets such as 95.2% on GIST1M and 94.1% on T2I-1B. This validates that the non-contiguous slab architecture introduces no precision loss. Second, SIVF maintains its throughput advantage even as the search depth increases to target high recall regimes. The slab management layer ensures high memory coalescing efficiency during vector traversal, which effectively masks the latency inherent in pointer chasing. Third, the consistent gap between the SIVF and baseline curves across the entire recall spectrum suggests that the system successfully eliminates the traditional trade-off between index flexibility and search performance.

4.4 End-to-End Streaming Performance

4.4.1 Sliding window. We evaluate real-time applicability using a *Sliding Window* benchmark that mimics production streams. The system maintains a fixed active window (W) by ingesting a new batch (B) and evicting the oldest batch (B) at each step. We test on SIFT1M ($W = 200$K, $B = 10$K) and GIST1M ($W = 100$K, $B = 5$K).

As shown in Figure 10, the standard Faiss baseline suffers from high latency due to the CPU-GPU roundtrip required for index reconstruction, causing system freezes of 355 ms (SIFT1M) and > 1.1 s (GIST1M) per update. In contrast, SIVF executes updates strictly in VRAM via slab-based management, reducing per-step latency to ≈ 2.2 ms (SIFT1M) and 4.2 ms (GIST1M). This yields speedups of 163× and 262×, respectively. Notably, the performance gap widens with dimensionality (GIST1M), where the baseline is bottlenecked by PCIe saturation. SIVF eliminates this bottleneck, maintaining single-digit millisecond latency and transforming the GPU IVF index into a real-time streaming engine.

4.4.2 Long-term streaming. To validate SIVF's stability under long-term streaming, we conducted a fine-grained benchmark with a reduced batch size of $B = 1,000$ over 1,000 continuous steps. As

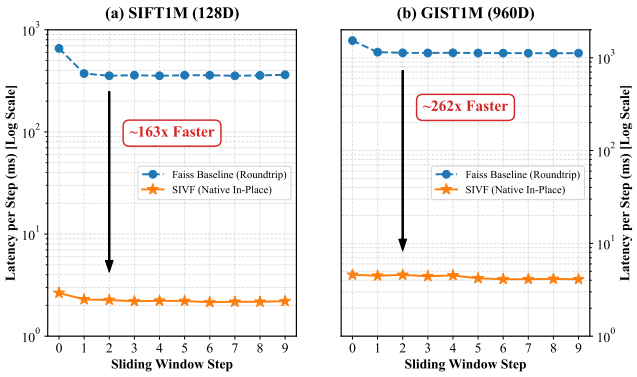


Figure 10: End-to-End Streaming Performance.

Table 1: Tail latency of deletion operations over 1,000 streaming steps (Batch Size $B = 1,000$).

Dataset	Dim	Avg (ms)	P99 (ms)	Max (ms)
SIFT1M	128	0.08	0.10	0.58
GIST1M	960	0.08	0.10	0.53

Table 2: Search latency stability under mixed workload.

Dataset	Dim	Avg (ms)	P99 (ms)
SIFT1M	128	0.25	0.41
GIST1M	960	0.69	0.78

shown in Table 1, the results confirm two critical characteristics. First, the latency scales linearly: reducing the batch size by 10× (compared to the standard batching in Fig. 7) reduces the average latency proportionally (from ≈ 0.89 ms to ≈ 0.08 ms), confirming the O(1) nature of our bitmap operations. Second, the performance is exceptionally stable: the 99th percentile (P99) latency is nearly identical to the average (0.10 ms vs 0.08 ms) for both SIFT1M (128D) and GIST1M (960D). This demonstrate that SIVF's lock-free retry mechanism introduces negligible jitter even under continuous churn, and its performance remains strictly dimension-agnostic.

4.4.3 Mixed operations. We evaluated search stability by interleaving query batches within the sliding window loop (Insert → Search → Delete). The results confirm that SIVF maintains sub-millisecond retrieval latency even under continuous mutation. On SIFT1M, the 99th percentile (P99) search latency was 0.41 ms (Avg: 0.25 ms). Notably, on the high-dimensional GIST1M, the system exhibited negligible jitter with a P99 of 0.78 ms (Avg: 0.69 ms). This stability verifies that our slab-based memory management prevents the fragmentation-induced performance degradation typical of dynamic GPU indices.

4.5 Hardware Utilization

4.5.1 Computation. To understand the micro-architectural drivers of the observed performance divergence, we profiled the GPU execution pipeline during the streaming update phase using NVIDIA Nsight Systems and Nsight Compute. Table 3 presents the breakdown of the total execution time. The baseline implementation

Table 3: Breakdown of GPU time during streaming updates.

Operation Category	GPU IVF	SIVF (Ours)
Data Transfer (PCIe)	53.2%	< 1.0%
Memory Mgmt (`malloc`/`free`)	39.2%	< 1.0%
Compute (Active Kernels)	3.2%	95.0%
Other Overheads	4.4%	≈ 4.0%

reveals a severe resource mismatch: 53.2% of the runtime is consumed by host-to-device data transfer (`cudaMemcpyAsync`) via the PCIe bus, and an additional 39.2% is spent on dynamic memory management overheads (`cudaMalloc` or `cudaFree`). Consequently, the actual GPU compute units (SMs) remain idle for over 96% of the update cycle, with only 3.2% of time utilized for kernel execution. In contrast, SIVF's GPU-resident architecture eliminates these PCIe roundtrips and OS-level allocations. As a result, the execution profile is transformed: 95.0% of the time is dedicated to active kernel execution, confirming that SIVF successfully shifts the workload from being I/O-bound to compute-bound.

Further profiling with Nsight Compute confirms that SIVF saturates on-device resources. During the ingestion phase on SIFT1M, the quantization kernels achieve a Streaming Multiprocessor (SM) utilization of 49.0%, indicating a healthy compute-bound workload. Simultaneously, the slab selection kernels exhibit an effective memory bandwidth of 150.3 GB/s. This demonstrates that despite the irregular memory access patterns inherent to linked-slab traversals, SIVF's warp-aligned design ($C = 32$) maintains sufficient coalescing to utilize a significant fraction of the device's memory bandwidth, a property unattainable by pointer-based CPU data structures.

4.5.2 Memory Efficiency and Scalability. A common concern with dynamic graph-based or list-based indices is the potential for memory bloating due to structural overhead, such as pointers and metadata, as well as fragmentation. To quantify the space complexity of SIVF, we conducted a memory footprint analysis comparing the allocated VRAM usage of SIVF against a theoretically compact array baseline across varying dataset sizes ranging from 100K to 1M vectors. Figure 11 illustrates the memory growth trends for both SIFT1M and GIST1M. The results demonstrate that SIVF exhibits deterministic linear scalability with negligible structural overhead. For SIFT1M ($d = 128$), the overhead stabilizes at 0.77%, while for the high-dimensional GIST1M ($d = 960$), it is merely 0.10%. This efficiency stems from our coarse-grained slab design, where the metadata cost (128-byte header) is amortized over 32 vectors.

We verified the efficacy of our memory reclamation strategy. In a stress test (not shown in the figure) involving the deletion of 50% of the dataset followed by immediate re-insertion, SIVF maintained a constant memory footprint without triggering OS-level deallocation. The deletion operation completed in sub-millisecond time per batch (e.g., 4.04 ms for 100K GIST vectors), confirming that memory slots are logically reclaimed via bitmap toggling and immediately available for reuse. This zero-cost reclamation ensures that SIVF can sustain long-running streaming workloads without suffering from memory leaks or fragmentation-induced bloat.

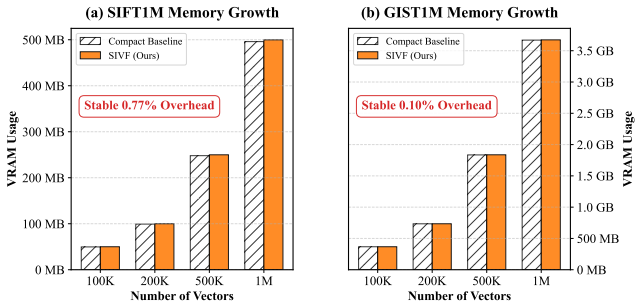


Figure 11: Memory Efficiency and Scalability.

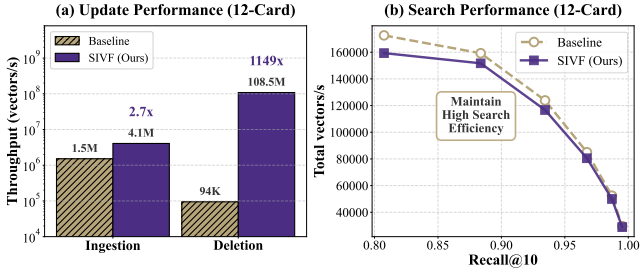


Figure 12: Distributed performance on the 12-GPU cluster (DINO10B dataset).

Table 4: Add throughput (K vec/s) and delete latency (ms).

Method	SIFT1M		T2I-1B		GIST1M	
	Add	Delete	Add	Delete	Add	Delete
Faiss Flat [7]	9,308	833	5,957	1,163	1,907	1,040
nuVS CAGRA [39]	305	3,030	259	3,773	97.1	10,215
HNSW [35]	25.1	39,835	25.8	39,141	0.6	334,113
NSG [9]	3.7	266,224	3.2	314,315	0.7	278,101
LSH [5]	823	13.7	461	16.4	38.4	8.5
SIVF (this work)	**2,229**	**0.95**	**5,183**	**1.52**	**452**	**1.31**

Figure 13: Scalability across a 4-node cluster of 12 GPUs.

4.6 Mainstream Indexes for Streaming Vectors

To position SIVF within the broader indexing landscape, we benchmarked it against five representative non-IVF architectures: GPU CAGRA [39] (graphs), Faiss GPU Flat (brute-force baseline, i.e., no index), HNSW [35] (dynamic graphs), (3) LSH [5] (legacy hash-based approaches), and (4) NSG [9] (static graphs). Table 4 summarizes ingestion throughput and deletion latency across three datasets: SIFT1M, T2I-1B, and GIST1M.

Table 4 reports the results of vector ingestion and deletion in a streaming context across three datasets. CPU-based graph indices (HNSW, NSG) suffer catastrophic performance degradation on high-dimensional data, dropping to merely 600 vec/s on GIST1M due to cache thrashing. They lack native support for deletion, necessitating full index reconstruction that incurs prohibitive latencies (e.g., over 330 seconds for HNSW on GIST1M). While GPU Flat achieves high ingestion throughput by bypassing indexing overhead, its deletion latencies remain high (> 1s) due to $O(N)$ data compaction and CPU-GPU synchronization. Notably, the state-of-the-art GPU graph index, CAGRA, exhibits severe limitations in streaming scenarios; enforcing true physical deletion to prevent memory leaks necessitates reconstruction, causing latencies to spike to over 10 seconds. In contrast, SIVF emerges as the only architecture enabling holistic stream processing: it sustains GPU-class ingestion rates (e.g., 452K vec/s on GIST1M, 4.6× faster than CAGRA) while providing millisecond eviction with immediate slot reuse (via bitmap toggling), and slab-level reclamation when a slab becomes empty.

4.7 Scalability on Multi-Node GPU Clusters

We evaluated the scalability performance of SIVF on the Chameleon TACC site [24]. Each node is equipped with dual-socket Intel Xeon E5 CPUs running at 2.00 GHz and 128 GB of RAM. The compute power is provided by NVIDIA Tesla P100 GPUs. The experimental environment leverages a total of 12 GPUs distributed across 4 physical nodes (4 GPUs each on `c11-01` and `c11-04`, and 2 GPUs each on `c11-03` and `c11-22`).

We evaluate SIVF against the primary baseline (Faiss GPU IVF-Flat) using the 1024-dimensional DINO10B dataset [1] on the 12-GPU cluster. Figure 12(a) highlights SIVF's efficiency in handling data updates. For ingestion, SIVF achieves a 2.69× speedup over the baseline (4.07M vectors/s vs. 1.51M vectors/s). The most significant performance gap is observed in deletion operations. SIVF achieves a 1150× speedup, processing over 108M deletions per second compared to the baseline's 94K. Figure 12(b) plots the Pareto frontier of Recall@10 versus throughput. SIVF exhibits zero recall loss and retains over 95% of the baseline's throughput.

Figure 13 shows near-linear scalability for 128-dimensional vectors. On 12 GPUs, ingestion peaks at 45.5 million vec/s, and search reaches 30.1K vectors/s (95% efficiency relative to the single-GPU baseline). Deletion exhibits super-linear scaling, achieving 103.2 million vectors/s. This efficiency boost stems from the compute-bound nature of the workload: expanding the cluster increases aggregate GPU cache capacity and reduces per-device contention for bitmap synchronization, thereby outperforming linear projections.

5 Conclusion

This work addresses the immutability issues in GPU-accelerated Inverted File (IVF) indices caused by static memory layouts. We present SIVF, a GPU-native index enabling high-velocity, in-place updates via slab-based memory management. By leveraging a lock-free update mechanism and optimized slab traversal, SIVF effectively masks the synchronization overhead typical of dynamic indexing, allowing the system to fully exploit hardware parallelism. Evaluation on a 12-GPU cluster demonstrates that SIVF achieves near perfect linear scalability, reaching 4.07M vectors/s for ingestion and 108.5M vectors/s for deletion. Furthermore, SIVF maintains high-accuracy search performance at 159.3K queries/s while keeping memory overhead negligible.

References

[1] Caron, M., Touvron, H., Misra, I., Jegou, H., Mairal, J., Bojanowski, P., and Joulin, A. Emerging properties in self-supervised vision transformers. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)* (2021), pp. 9630–9640.

[2] Chatzakis, M., Papakonstantinou, Y., and Palpanas, T. Darth: Declarative recall through early termination for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 4 (Sept. 2025).

[3] Chu, G., He, Y., Dong, L., Ding, Z., Chen, D., Bai, H., Wang, X., and Hu, C. Efficient algorithm design of optimizing spmv on gpu. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 115–128.

[4] Chung, J. W., Lin, H., and Zhao, W. Locality-sensitive indexing for graph-based approximate nearest neighbor search. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (New York, NY, USA, 2025), SIGIR '25, Association for Computing Machinery, p. 2418–2428.

[5] Datar, M., Immorlica, N., Indyk, P., and Mirrokni, V. S. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the Twentieth Annual Symposium on Computational Geometry* (New York, NY, USA, 2004), SCG '04, Association for Computing Machinery, p. 253–262.

[6] Deng, L., Chen, P., Zeng, X., Wang, T., Zhao, Y., and Zheng, K. Efficient data-aware distance comparison operations for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow. 18*, 3 (Nov. 2024), 812–821.

[7] Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., and Jégou, H. The faiss library.

[8] Engels, J., Landrum, B., Yu, S., Dhulipala, L., and Shun, J. Approximate nearest neighbor search with window filters. In *Forty-first International Conference on Machine Learning* (2024).

[9] Fu, C., Xiang, C., Wang, C., and Cai, D. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proc. VLDB Endow. 12*, 5 (Jan. 2019), 461–474.

[10] Gao, J., Gou, Y., Xu, Y., Yang, Y., Long, C., and Wong, R. C.-W. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[11] Gao, J., and Long, C. Rabitq: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proc. ACM Manag. Data 2*, 3 (May 2024).

[12] Gong, Z., Zeng, Y., and Chen, L. Accelerating approximate nearest neighbor search in hierarchical graphs: Efficient level navigation with shortcuts. *Proc. VLDB Endow. 18*, 10 (June 2025), 3518–3530.

[13] Gou, Y., Gao, J., Xu, Y., and Long, C. Symphonyqg: Towards symphonious integration of quantization and graph for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 1 (Feb. 2025).

[14] Hua, Z., Mo, Q., Yao, Z., Cui, L., Liu, X., Wang, G., Wei, Z., Liu, X., Tang, T., Liu, S., and Qu, L. Dynamically detect and fix hardness for efficient approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[15] Hui, X., Xu, Y., Guo, Z., and Shen, X. Esg: Pipeline-conscious efficient scheduling of dnn workflows on serverless platforms with shareable gpus. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 42–55.

[16] Hui, X., Xu, Y., and Shen, X. Fluidfaas: A dynamic pipelined solution for serverless computing with strong isolation-based gpu sharing. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2025), HPDC '25, Association for Computing Machinery.

[17] Hwang, S., Lee, E., Oh, H., and Yi, Y. Fasop: Fast yet accurate automated search for optimal parallelization of transformers on heterogeneous gpu clusters. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 253–266.

[18] Indyk, P., and Xu, H. Worst-case performance of popular approximate nearest neighbor search implementations: Guarantees and limitations. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.

[19] Jääsaari, E., Hyvönen, V., and Roos, T. Lorann: Low-rank matrix factorization for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024* (2024), A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds.

[20] Jiang, M., Yang, Z., Zhang, F., Hou, G., Shi, J., Zhou, W., Li, F., and Wang, S. Digra: A dynamic graph indexing for approximate nearest neighbor search with range filter. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[21] Johnson, J., Douze, M., and Jégou, H. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data 7*, 3 (2021), 535–547.

[22] Jung, S., Park, Y., Lee, H., Oh, Y. H., and Lee, J. W. Angular distance-guided neighbor selection for graph-based approximate nearest neighbor search. In *Proceedings of the ACM on Web Conference 2025* (New York, NY, USA, 2025), WWW '25, Association for Computing Machinery, p. 4014–4023.

[23] Jégou, H., Douze, M., and Schmid, C. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence 33*, 1 (2011), 117–128.

[24] Keahey, K., Anderson, J., Zhen, Z., Riteau, P., Ruth, P., Stanzione, D., Cevik, M., Colleran, J., Gunawi, H. S., Hammock, C., Mambretti, J., Barnes, A., Halbach, F., Rocha, A., and Stubbs, J. Lessons learned from the chameleon testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association, July 2020.

[25] Laut, S., Borrell, R., and Casas, M. Extending sparse patterns to improve inverse preconditioning on gpu architectures. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 200–213.

[26] Leyba, K., Hofmeyr, S., Forrest, S., Cannon, J., and Moses, M. Simcov-gpu: Accelerating an agent-based model for exascale. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 322–333.

[27] Li, M., Yan, X., Lu, B., Zhang, Y., Cheng, J., and Ma, C. Attribute filtering in approximate nearest neighbor search: An in-depth experimental study. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[28] Li, X., Laguna, I., Fang, B., Swirydowicz, K., Li, A., and Gopalakrishnan, G. Design and evaluation of gpu-fpx: A low-overhead tool for floating-point exception detection in nvidia gpus. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 59–71.

[29] Li, Z., Ke, X., Zhu, Y., Yu, B., Zheng, B., and Gao, Y. Scalable graph indexing using gpus for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[30] Li, Z., Ke, X., Zhu, Y., Yu, B., Zheng, B., and Gao, Y. Scalable graph indexing using gpus for approximate nearest neighbor search. In *Proceedings of the 2026 International Conference on Management of Data (SIGMOD)* (2026).

[31] Liang, A., Zhang, P., Yao, B., Chen, Z., Song, Y., and Cheng, G. Unify: Unified index for range filtered approximate nearest neighbors search. *Proc. VLDB Endow. 18*, 4 (Dec. 2024), 1118–1130.

[32] Liu, M., Zhong, S., Yang, Q., Han, Y., Liu, X., and Ma, Y. Webanns: Fast and efficient approximate nearest neighbor search in web browsers. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (New York, NY, USA, 2025), SIGIR '25, Association for Computing Machinery, p. 2483–2492.

[33] Lu, D., Feng, S., Zhou, J., Solleza, F., Schwarzkopf, M., and Çetintemel, U. Vectraflow: Integrating vectors into stream processing. In *Proceedings of the 2025 Conference on Innovative Data Systems Research (CIDR)* (Jan. 2025).

[34] Lu, K., Xiao, C., and Ishikawa, Y. Probabilistic routing for graph-based approximate nearest neighbor search. In *Forty-first International Conference on Machine Learning* (2024).

[35] Malkov, Y. A., and Yashunin, D. A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Trans. Pattern Anal. Mach. Intell. 42*, 4 (Apr. 2020), 824–836.

[36] Maurya, A., Rafique, M. M., Tonellot, T., AlSalem, H. J., Cappello, F., and Nicolae, B. Gpu-enabled asynchronous multi-level checkpoint caching and prefetching. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 73–85.

[37] Miao, D., Laguna, I., and Rubio-González, C. Floatguard: Efficient whole-program detection of floating-point exceptions in amd gpus. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2025), HPDC '25, Association for Computing Machinery.

[38] Mohoney, J., Sarda, D., Tang, M., Chowdhury, S. R., Pacaci, A., Ilyas, I. F., Rekatsinas, T., and Venkataraman, S. Quake: adaptive indexing for vector search. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation* (USA, 2025), OSDI '25, USENIX Association.

[39] Ootomo, H., Naruse, A., Nolet, C., Wang, R., Feher, T., and Wang, Y. CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs . In *2024 IEEE 40th International Conference on Data Engineering (ICDE)* (Los Alamitos, CA, USA, May 2024), IEEE Computer Society, pp. 4236–4247.

[40] Peng, Z., Qiao, M., Zhou, W., Li, F., and Deng, D. Dynamic range-filtering approximate nearest neighbor search. *Proc. VLDB Endow. 18*, 10 (June 2025), 3256–3268.

[41] Peng, Z., Thomadakis, P., Pienaar, J., and Kestor, G. Liteform: Lightweight and automatic format composition for sparse matrix-matrix multiplication on gpus. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2025), HPDC '25, Association for Computing Machinery.

[42] Qiu, R., and Tang, J. Efficient approximate nearest neighbor search via hemisphere centroids graph. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[43] simhadri, H. V., Aumüller, M., Douze, M., Baranchuk, D., Ingber, A., Liberty, E., Williams, G., Landrum, B., Manohar, M. D., Karjikar, M., Dhulipala, L., Chen, M., Chen, Y., Ma, R., Zhang, K., Cai, Y., Shi, J., Zheng, W., Chen, Y., Yin, J., and Huang, B. Results of the big ANN: NeurIPS'23 competition. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track* (2025).

[44] Sun, P., Simcha, D., Dopson, D., Guo, R., and Kumar, S. SOAR: improved indexing for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.

[45] Tran, A., Laguna, I., and Gopalakrishnan, G. Fpboxer: Efficient input-generation for targeting floating-point exceptions in gpu programs. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 83–93.

[46] Voruganti, S., and Özsu, M. T. Mirage-anns: Mixed approach graph-based indexing for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[47] Wang, M., Lv, L., Xu, X., Wang, Y., Yue, Q., and Ni, J. An efficient and robust framework for approximate nearest neighbor search with attribute constraint. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.

[48] Wang, Z., Zhang, J., and Hu, W. Wow: A window-to-window incremental index for range-filtering approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[49] Wei, J., Lee, X., Liao, Z., Palpanas, T., and Peng, B. Subspace collision: An efficient and accurate framework for high-dimensional approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 1 (Feb. 2025).

[50] Xu, Q., Zhang, F., Li, C., Cao, L., Chen, Z., Zhai, J., and Du, X. Harmony: A scalable distributed vector database for high-throughput approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 4 (Sept. 2025).

[51] Yandex, A. B., and Lempitsky, V. Efficient indexing of billion-scale datasets of deep descriptors. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016), pp. 2055–2063.

[52] Yang, M., Cai, Y., and Zheng, W. CSPG: crossing sparse proximity graphs for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024* (2024), A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds.

[53] Zhang, B., Tian, J., Di, S., Yu, X., Feng, Y., Liang, X., Tao, D., and Cappello, F. Fz-gpu: A fast and high-ratio lossy compressor for scientific computing applications on gpus. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 129–142.

[54] Zhang, F., Jiang, M., Hou, G., Shi, J., Fan, H., Zhou, W., Li, F., and Wang, S. Efficient dynamic indexing for range filtered approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[55] Zhang, Z., Liu, F., Huang, G., Liu, X., and Jin, X. Fast vector query processing for large datasets beyond GPU memory with reordered pipelining. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)* (Santa Clara, CA, Apr. 2024), USENIX Association, pp. 23–40.

[56] Zhong, X., Li, H., Jin, J., Yang, M., Chu, D., Wang, X., Shen, Z., Jia, W., Gu, G., Xie, Y., Lin, X., Shen, H. T., Song, J., and Cheng, P. Vsag: An optimized search framework for graph-based approximate nearest neighbor search. *Proc. VLDB Endow. 18*, 12 (Aug. 2025), 5017–5030.